Refactoring Strategies Guide

Loan Management System Edition

Masoud Bahrmai

http://MasoudBahrami.com

PART 1: INTRODUCTION

Refactoring vs Rewriting

The Critical Distinction

Before we dive into specific strategies, we must understand the fundamental difference between refactoring and rewriting. This distinction determines everything about how we approach code improvement.

Refactoring

Definition: Refactoring is a controlled change to the internal structure of software that preserves its externally observable behavior.

Key Characteristics:

- **Changes internal structure only**
- **Preserves all external behavior**
- **Small, incremental steps**
- **Low risk**
- **Tests protect behavior**

- **Hours to days to complete**

Example from Loan Management System:

// BEFORE: Inline request creation

```
public async Task ProcessLoanAsync(Loan loan)
{
    var request = new ExternalLoanRequest
    {
        CustomerId = loan.CustomerId,
        Amount = loan.Amount,
        Term = loan.Term,
        Type = loan.Type
    };

    var response = await _httpClient.PostAsJsonAsync("/api/loans", request);
    // ... rest of method
}
```

// AFTER: Extracted method (behavior unchanged)

```
public async Task ProcessLoanAsync(Loan loan)
{
    var request = BuildRequest(loan);
    var response = await _httpClient.PostAsJsonAsync("/api/loans", request);
    // ... rest of method
}

private ExternalLoanRequest BuildRequest(Loan loan)
{
    return new ExternalLoanRequest
    {
        CustomerId = loan.CustomerId,
        Amount = loan.Amount,
        Term = loan.Term,
        Type = loan.Type
    };
}
```

Result: The same request is built, the same API call is made, the same response is returned. Behavior is identical. Structure is improved.

Rewriting

Definition: Rewriting is replacing the existing implementation with a new one, often with a new design, architecture, or interpretation of requirements.

Key Characteristics:

- **Replaces entire implementation**
- **Behavior may change**
- **Large, all-at-once changes**
- **High risk**
- **New tests needed**
- **Weeks to months to complete**

Example from Loan Management System:

Deleting *LoanProcessingEngine.cs,* introducing a completely new application architecture, replacing the Excel library, replacing the HTTP client, introducing a message broker, changing the retry mechanism, redesigning the loan domain model, and implementing business rules again from interpretation of existing behavior.

Result: A completely new system. Behavior may or may not match the old system.

The 20% Rule

There is a common saying in the software industry: "If a system is more than 20% dead, don't refactor it. Rewrite it."

What This Actually Means:

This is NOT a magical percentage. It's about change scope and risk concentration.

Better Interpretation:

Keep the refactoring scope small enough that one team can:

- **Understand the affected behavior**
- **Establish safety**
- **Make the structural change**
- **Verify the result**

Example:

Instead of refactoring the entire LoanProcessingEngine (600+ lines), isolate one coherent behavior:

LoanProcessingEngine

|

+-- Excel reading

|

+-- Validation

|

+-- Approval ← Refactor this first

|

+-- Cancellation

|

+-- External API

|

+-- Retry

|

+-- Audit

|

+--

When to Refactor vs Rewrite

Factor	Refactor	Rewrite
Business rules	Still valid	Obsolete
Behavior understanding	Good	Poor
Test coverage	Some	None
Architecture	Poor but salvageable	Fundamentally broken
Technology	Supported	Unsupported
Requirements	Stable	Changed
Domain model	Valuable	Wrong
Behavior knowledge	In code	Lost
Component size	Small to medium	Large
Technical debt	Manageable	Crippling

Refactoring Flow

The 4-Step Process

1. UNDERSTAND

Before touching code, understand the system

- Build a behavior map

- Identify inputs, outputs, dependencies

- Look for undocumented assumptions

- Identify entry points and critical paths

2. CREATE SAFETY

Create a safety net before changing anything

- Characterization tests

- Golden Master tests

- Integration tests
- Identify boundaries and risks

3. REFACTOR

Make small, incremental structural changes

- One change at a time
- Use appropriate strategy
- Keep changes focused
- Commit frequently

4. VERIFY

Confirm behavior is preserved

- Run tests
- Compare outputs
- Review changes
- Validate with stakeholders

The Feedback Loop

Refactoring is not a linear process. It's a feedback loop:

Observe

Hypothesize

Change

Verify

Learn

Change again

This is especially important in legacy systems because our understanding of the system changes as we investigate it.

PART 2: WHERE DO I START?

Strategy 1: Top-Down Refactoring

Definition

Top-Down refactoring starts at the highest level of abstraction and works downward. The goal is to make the business workflow visible first, then refine the implementation details.

Core Question

"Can I make the business workflow visible first?"

When to Use

- **The business workflow is understood but obscured by implementation details**
- **The main method is difficult to read**
- **Lower-level details obscure the business sequence**
- **You need to communicate the code's behavior to the team**

- **The high-level code has the most business impact**
- **You need to understand the overall flow before refactoring details**

When NOT to Use

- **The business workflow is unclear or disputed**
- **Lower-level infrastructure is blocking understanding**
- **You're not confident in your understanding of the workflow**
- **The system has no clear separation between layers**

Example: Loan Management System

Problem: ProcessLoanRequestsAsync is 80+ lines. The business workflow is buried in details.

BEFORE:

```
public async Task ProcessLoanRequestsAsync()
{
    LogStep("Starting Loan Approval Request Processing");
    LogInfo($"Excel file path: {_loanApprovalExcelPath}");

    try
    {
        if (!File.Exists(_loanApprovalExcelPath))
        {
            LogError($"Excel file not found at: {_loanApprovalExcelPath}");
            return;
        }

        var allRecords = ReadExcelData(_loanApprovalExcelPath,
RequestType.Approval);
        var processedApplicants = await GetProcessedApplicantsAsync();
        var retryTracker = await GetRetryTrackerAsync();

        var recordsToProcess = allRecords
            .Where(r => !processedApplicants.Contains(r.ChassisNumber,
StringComparer.OrdinalIgnoreCase))
            .ToList();

        if (!recordsToProcess.Any())
        {
            LogWarning("All applicants have already been processed
successfully.");
```

```
            return;
        }

        var recordsWithRetryInfo = recordsToProcess
            .Select(record => new
            {
                Record = record,
                RetryCount = retryTracker.TryGetValue(record.ChassisNumber,
out var count) ? count : 0,
                ShouldProcess =
!retryTracker.TryGetValue(record.ChassisNumber, out var retryCount) ||
retryCount < _maxRetryAttempts
            })
            .ToList();

        var recordsToActuallyProcess = recordsWithRetryInfo.Where(r =>
r.ShouldProcess).Select(r => r.Record).ToList();
        var recordsExceedingRetries = recordsWithRetryInfo.Where(r =>
!r.ShouldProcess).Select(r => r.Record).ToList();

        if (recordsExceedingRetries.Any())
        {
            LogWarning($"Skipping {recordsExceedingRetries.Count} applicants
exceeding max retry attempts ({_maxRetryAttempts}):");
            foreach (var record in recordsExceedingRetries)
            {
                LogWarning($"  - {record.ChassisNumber} (Attempts:
{retryTracker[record.ChassisNumber]})");
            }
            var failedResults = recordsExceedingRetries.Select(r => new
ProcessingResult(r.ChassisNumber, false, $"Exceeded max retry attempts
({_maxRetryAttempts})")).ToList();
            await ArchiveToDeadLetterAsync(failedResults,
RequestType.Approval);
        }

        if (!recordsToActuallyProcess.Any())
        {
            LogWarning("No applicants to process (all have exceeded max retry
attempts).");
            return;
        }

        LogSuccess($"Processing {recordsToActuallyProcess.Count} applicants
(skipping {processedApplicants.Count} already processed,
{recordsExceedingRetries.Count} exceeded retry limit)");

        var results = await ProcessRecordsAsync(recordsToActuallyProcess,
RequestType.Approval);
        await UpdateRetryTrackerAsync(results.Where(r => !r.IsSuccess));
        await MarkApplicantsAsProcessedAsync(results.Where(r =>
r.IsSuccess).Select(r => r.ChassisNumber));
        await ArchiveToDeadLetterAsync(results.Where(r => !r.IsSuccess),
RequestType.Approval);
```

```
        DisplaySummary(results, "Loan Approval");
    }
    catch (Exception ex)
    {
        LogError($"Error processing loan approval requests: {ex.Message}");
        LogDebug($"Stack trace: {ex.StackTrace}");
        throw;
    }
}
```

AFTER - Top-Down Refactoring:

```
public async Task ProcessLoanRequestsAsync(RequestType requestType)
{
    try
    {
        LogStep($"Starting {requestType.GetDisplayName()} Processing");
        LogInfo($"Excel file path: {GetExcelPath(requestType)}");

        // 1. Prepare records
        var (toProcess, skipped) = await PrepareRecordsAsync(requestType);

        if (!toProcess.Any())
        {
            LogWarning("No records to process");
            return;
        }

        LogSuccess($"Processing {toProcess.Count} applicants (skipped
{skipped.Count})");

        // 2. Process records
        var results = await ProcessRecordsAsync(toProcess, requestType);

        // 3. Report results
        await ReportResultsAsync(results, requestType);
        DisplaySummary(results, requestType.GetDisplayName());
    }
    catch (Exception ex)
    {
        LogError($"Error processing {requestType.GetDisplayName()} requests:
{ex.Message}");
        throw;
    }
}

private async Task<(List<LoanRequestRecord> ToProcess,
List<LoanRequestRecord> Skipped)>
    PrepareRecordsAsync(RequestType requestType)
{
    var allRecords = await LoadRecordsAsync(requestType);
    var processed = await GetProcessedApplicantsAsync();
    var retryTracker = await GetRetryTrackerAsync();
```

```
    return FilterRecords(allRecords, processed, retryTracker);
}

private async Task<List<ProcessingResult>> ProcessRecordsAsync(
    List<LoanRequestRecord> records,
    RequestType requestType)
{
    var results = new List<ProcessingResult>();
    var total = records.Count;
    var processed = 0;

    foreach (var record in records)
    {
        processed++;
        LogProgress($"Processing {processed}/{total}:
{record.ChassisNumber}");

        try
        {
            var result = await ProcessSingleRecordAsync(record, requestType);
            results.Add(result);

            if (result.IsSuccess)
                LogSuccess($"✓ {record.ChassisNumber} processed
successfully");
            else
                LogError($"✗ {record.ChassisNumber} failed:
{result.Message}");
        }
        catch (Exception ex)
        {
            LogError($"✗ {record.ChassisNumber} threw exception:
{ex.Message}");
            results.Add(new ProcessingResult(record.ChassisNumber, false,
$"Exception: {ex.Message}"));
        }
    }

    return results;
}

private async Task ReportResultsAsync(List<ProcessingResult> results,
RequestType requestType)
{
    var failedResults = results.Where(r => !r.IsSuccess).ToList();
    var successResults = results.Where(r => r.IsSuccess).ToList();

    if (failedResults.Any())
    {
        await UpdateRetryTrackerAsync(failedResults);
        await ArchiveToDeadLetterAsync(failedResults, requestType);
    }

    if (successResults.Any())
    {
```

```
        await MarkApplicantsAsProcessedAsync(successResults.Select(r =>
r.ChassisNumber));
    }
}
```

Why It Works:

The business story becomes visible. The code reads like a narrative:

1. **Prepare records**
2. **Process records**
3. **Report results**

Each step is now a separate, focused method.

The Danger:

You can extract methods without actually reducing coupling. You end up with:

```
ProcessLoan()
  -> Step1()
  -> Step2()
  -> Step3()
  -> Step4()
  -> Step5()
```

But every method still accesses the same 15 fields. You've created method fragmentation, not good design.

Strategy 2: Bottom-Up Refactoring

Definition

Bottom-Up refactoring starts with lower-level implementation details and gradually moves upward toward the business workflow.

Core Question

"Can I clean the lower-level dependency that is blocking me?"

When to Use

- **Lower-level code is highly reusable**
- **Lower-level code is relatively isolated**
- **Lower-level code is technically messy but behaviorally well understood**
- **Lower-level code is easy to test**
- **Lower-level code is creating noise for higher-level code**
- **The lower layer is blocking understanding of the higher layer**

When NOT to Use

- **The lower-level code is rarely used or unimportant**
- **The business workflow is the main problem**
- **The lower-level code is well-understood and not causing issues**
- **You're cleaning infrastructure just because it's ugly, not because it's blocking progress**

Example: Loan Management System

Problem: ExternalLoanService has mixed responsibilities. HTTP, normalization, retry, parsing all in one class.

BEFORE:

```
public class ExternalLoanService
{
    private const string BASE_URL =
@"http://api.internal.loan.local/api/services/LoanProcessing/";
    private const string AUTH_TOKEN = "Basic
dXNlcl9sb2FuOmJMb2NrfG5HXzE0MDMwNDAz";
    private const string API_KEY = "L98m64KHjl98G";
    private const int TIMEOUT_SECONDS = 60;

    public async Task<ServiceResponse<LoanOperationResult>>
ApproveLoanAsync(LoanApprovalRequest request)
    {
        var client = CreateClient("ApproveLoan");
        var restRequest = CreateRequest(Method.POST);

        if (request.requestDate.HasValue)
        {
            request.requestDate = new DateTime(
                request.requestDate.Value.Year,
```

```
                request.requestDate.Value.Month,
                request.requestDate.Value.Day,
                request.requestDate.Value.Hour,
                request.requestDate.Value.Minute,
                request.requestDate.Value.Second
            );
        }

        if (!string.IsNullOrEmpty(request.phoneNumber))
        {
            request.phoneNumber = "0" + request.phoneNumber;
        }

        restRequest.AddParameter("application/json",
JsonConvert.SerializeObject(request), ParameterType.RequestBody);
        return ExecuteRequest<LoanOperationResult>(client, restRequest,
"ApproveLoan");
    }

    // Similar methods for AddLoan, RemoveLoan, CancelLoan...
    // All with the same pattern
}
```

BOTTOM-UP - Extract Retry Logic:

```
// Step 1: Extract retry logic
public interface IRetryHandler
{
    Task<T> ExecuteWithRetryAsync<T>(Func<Task<T>> operation);
}

public class RetryHandler : IRetryHandler
{
    private readonly int _maxAttempts;
    private readonly TimeSpan _initialDelay;

    public RetryHandler(int maxAttempts = 5, int initialDelaySeconds = 1)
    {
        _maxAttempts = maxAttempts;
        _initialDelay = TimeSpan.FromSeconds(initialDelaySeconds);
    }

    public async Task<T> ExecuteWithRetryAsync<T>(Func<Task<T>> operation)
    {
        int attempt = 0;
        while (attempt < _maxAttempts)
        {
            attempt++;
            try
            {
                return await operation();
            }
            catch (Exception)
            {
```

```
                if (attempt == _maxAttempts)
                    throw;

                var delay = TimeSpan.FromMilliseconds(
                    _initialDelay.TotalMilliseconds * Math.Pow(2, attempt)
                );
                await Task.Delay(delay);
            }
        }
        throw new InvalidOperationException("Retry logic failed");
    }
}
```

Step 2: Extract HTTP Client:

```
// Step 2: Extract HTTP communication
public interface ILoanApiClient
{
    Task<HttpResponseMessage> SendAsync(string endpoint, object request);
}

public class LoanApiClient : ILoanApiClient
{
    private readonly HttpClient _httpClient;
    private readonly string _apiKey;
    private readonly string _authToken;

    public LoanApiClient(HttpClient httpClient, IConfiguration configuration)
    {
        _httpClient = httpClient;
        _apiKey = configuration["LoanApi:ApiKey"];
        _authToken = configuration["LoanApi:AuthToken"];
    }

    public async Task<HttpResponseMessage> SendAsync(string endpoint, object
request)
    {
        var url = $"{endpoint}?key={_apiKey}";
        var content = new StringContent(
            JsonConvert.SerializeObject(request),
            Encoding.UTF8,
            "application/json"
        );

        using var requestMessage = new HttpRequestMessage(HttpMethod.Post,
url)
        {
            Content = content
        };
        requestMessage.Headers.Add("Authorization", _authToken);

        return await _httpClient.SendAsync(requestMessage);
    }
}
```

Step 3: Extract Request Normalizer:

```
// Step 3: Extract request normalization
public interface IRequestNormalizer
{
    T Normalize<T>(T request) where T : class;
}

public class LoanRequestNormalizer : IRequestNormalizer
{
    public T Normalize<T>(T request) where T : class
    {
        if (request is LoanApprovalRequest approvalRequest)
        {
            return NormalizeApprovalRequest(approvalRequest) as T;
        }

        if (request is LoanCancellationRequest cancellationRequest)
        {
            return NormalizeCancellationRequest(cancellationRequest) as T;
        }

        return request;
    }

    private LoanApprovalRequest NormalizeApprovalRequest(LoanApprovalRequest
request)
    {
        if (request.requestDate.HasValue)
        {
            request.requestDate = new DateTime(
                request.requestDate.Value.Year,
                request.requestDate.Value.Month,
                request.requestDate.Value.Day,
                request.requestDate.Value.Hour,
                request.requestDate.Value.Minute,
                request.requestDate.Value.Second
            );
        }

        if (!string.IsNullOrEmpty(request.phoneNumber) &&
!request.phoneNumber.StartsWith("0"))
        {
            request.phoneNumber = "0" + request.phoneNumber;
        }

        return request;
    }

    private LoanCancellationRequest
NormalizeCancellationRequest(LoanCancellationRequest request)
    {
        if (request.requestDate.HasValue)
        {
            request.requestDate = new DateTime(
```

```
                request.requestDate.Value.Year,
                request.requestDate.Value.Month,
                request.requestDate.Value.Day,
                request.requestDate.Value.Hour,
                request.requestDate.Value.Minute,
                request.requestDate.Value.Second
            );
        }

        if (!string.IsNullOrEmpty(request.phoneNumber) &&
!request.phoneNumber.StartsWith("0"))
        {
            request.phoneNumber = "0" + request.phoneNumber;
        }

        return request;
    }
}
```

Step 4: Refactor ExternalLoanService:

```
// Step 4: Clean ExternalLoanService
public class ExternalLoanService
{
    private readonly ILoanApiClient _apiClient;
    private readonly IRequestNormalizer _normalizer;
    private readonly IRetryHandler _retryHandler;
    private readonly ILogger _logger;

    public ExternalLoanService(
        ILoanApiClient apiClient,
        IRequestNormalizer normalizer,
        IRetryHandler retryHandler,
        ILogger logger)
    {
        _apiClient = apiClient;
        _normalizer = normalizer;
        _retryHandler = retryHandler;
        _logger = logger;
    }

    public async Task<ServiceResponse<LoanOperationResult>> ApproveLoanAsync(
        LoanApprovalRequest request)
    {
        return await ExecuteOperationAsync(
            "ApproveLoan",
            "/loans/approve",
            request,
            async () =>
            {
                var normalized = _normalizer.Normalize(request);
                return await _apiClient.SendAsync("/loans/approve",
normalized);
```

```
            }
        );
    }

    public async Task<ServiceResponse<LoanOperationResult>> CancelLoanAsync(
        LoanCancellationRequest request)
    {
        return await ExecuteOperationAsync(
            "CancelLoan",
            "/loans/cancel",
            request,
            async () =>
            {
                var normalized = _normalizer.Normalize(request);
                return await _apiClient.SendAsync("/loans/cancel",
normalized);
            }
        );
    }

    private async Task<ServiceResponse<T>> ExecuteOperationAsync<T>(
        string operationName,
        string endpoint,
        object request,
        Func<Task<HttpResponseMessage>> operation)
    {
        try
        {
            _logger.LogDebug($"Executing {operationName}");

            var response = await _retryHandler.ExecuteWithRetryAsync(async ()
=>
            {
                var result = await operation();

                if (!result.IsSuccessStatusCode)
                {
                    var error = await result.Content.ReadAsStringAsync();
                    throw new HttpRequestException(
                        $"Request failed: {result.StatusCode} - {error}"
                    );
                }

                return result;
            });

            var content = await response.Content.ReadAsStringAsync();
            var data = JsonConvert.DeserializeObject<T>(content);
            return ServiceResponse<T>.FromSuccess((int)response.StatusCode,
data);
        }
        catch (Exception ex)
        {
            _logger.LogError($"Error in {operationName}: {ex.Message}");
            return ServiceResponse<T>.FromError(
```

```
                (int)HttpStatusCode.InternalServerError,
                $"Error in {operationName}: {ex.Message}"
            );
        }
    }
}
```

Why It Works:

Each component now has a single responsibility:

- **LoanApiClient handles HTTP communication**
- **RequestNormalizer handles normalization**
- **RetryHandler handles retry logic**
- **ExternalLoanService orchestrates the operations**

The Danger:

Bottom-Up can become an infrastructure-cleaning project. You can spend hours creating beautiful abstractions while the business logic remains a mess.

Bottom-Up is good when the lower layer is blocking understanding of the higher layer. Not simply because the lower layer is ugly.

Strategy 3: Beachhead Refactoring

Definition

Beachhead refactoring is the strategy of selecting one small, strategically valuable area and making it clean and well-bounded while leaving the rest of the legacy system untouched. Think of a military beachhead: you don't conquer the entire territory before establishing your first secure position.

Core Question

"Where can I create one clean, strategically valuable island?"

When to Use

- **The entire system is too risky to restructure at once**
- **You need to prove the value of refactoring to stakeholders**
- **You have limited time and need to focus on the most important area**
- **You need to establish a pattern for future refactoring**
- **You need to create a safe place to experiment with new approaches**

When NOT to Use

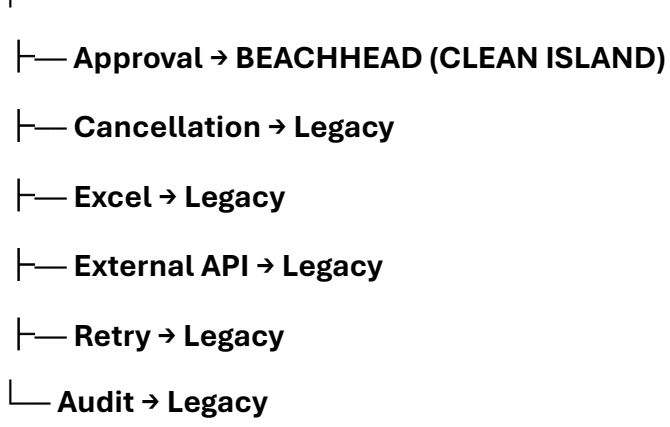
- **The system is small enough to refactor entirely**
- **You have stakeholder buy-in for larger refactoring**
- **The selected area is not strategically important**
- **The selected area is too complex for a focused effort**

Example: Loan Management System

Problem: The entire LoanProcessingEngine is a mess, but we can't refactor everything at once.

BEACHHEAD APPROACH:

LEGACY SYSTEM



Step 1: Create the Approval Beachhead:

```
// Create a clean, well-bounded Approval module
public interface ILoanApproval
{
    Task<ApprovalResult> ProcessAsync(ApprovalRequest request);
}

public class ApprovalRequest
{
    public ChassisNumber ChassisNumber { get; set; }
    public NationalId NationalId { get; set; }
    public PolicyId PolicyId { get; set; }
    public PlateNumber PlateNumber { get; set; }
    public Region Region { get; set; }
    public Branch Branch { get; set; }
    public LoanAmount Amount { get; set; }
    public DateTime ValidFrom { get; set; }
    public DateTime ValidTo { get; set; }
    public string Description { get; set; }
    public int ProcessingMode { get; set; }
}

public class ApprovalResult
{
    public bool IsSuccess { get; }
    public string Message { get; }
    public string ApplicantId { get; }
    public string ErrorCode { get; }

    private ApprovalResult(bool isSuccess, string message, string
applicantId, string errorCode)
    {
        IsSuccess = isSuccess;
        Message = message;
        ApplicantId = applicantId;
        ErrorCode = errorCode;
    }

    public static ApprovalResult Success(string applicantId, string message =
"Loan approved successfully")
        => new ApprovalResult(true, message, applicantId, null);

    public static ApprovalResult Failure(string message, string errorCode =
"APPROVAL_FAILED")
        => new ApprovalResult(false, message, null, errorCode);
}

public class LoanApproval : ILoanApproval
{
    private readonly IApprovalValidator _validator;
    private readonly IApprovalGateway _gateway;
    private readonly IApprovalAudit _audit;
    private readonly ILogger _logger;
```

```
    public LoanApproval(
        IApprovalValidator validator,
        IApprovalGateway gateway,
        IApprovalAudit audit,
        ILogger logger)
    {
        _validator = validator;
        _gateway = gateway;
        _audit = audit;
        _logger = logger;
    }

    public async Task<ApprovalResult> ProcessAsync(ApprovalRequest request)
    {
        _logger.LogInfo($"Processing approval for chassis:
{request.ChassisNumber}");

        try
        {
            // Step 1: Validate
            var validation = await _validator.ValidateAsync(request);
            if (!validation.IsValid)
            {
                _logger.LogWarning($"Validation failed:
{validation.Reason}");
                await _audit.LogFailureAsync(request, validation.Reason);
                return ApprovalResult.Failure(validation.Reason,
"VALIDATION_FAILED");
            }
```

// Step 2: Build gateway request

```
    var gatewayRequest = MapToGatewayRequest(request);


            // Step 3: Process through gateway
            var result = await _gateway.ApproveAsync(gatewayRequest);
            if (!result.IsSuccess)
            {
                _logger.LogError($"Gateway failed: {result.Message}");
                await _audit.LogFailureAsync(request, result.Message);
                return ApprovalResult.Failure(result.Message,
"GATEWAY_FAILED");
            }
```

// Step 4: Audit success

```
    await _audit.LogSuccessAsync(request, result);


            _logger.LogSuccess($"Approval completed for chassis:
{request.ChassisNumber}");
            return ApprovalResult.Success(result.ApplicantId);
```

```
        }
        catch (Exception ex)
        {
            _logger.LogError($"Unexpected error: {ex.Message}");
            await _audit.LogErrorAsync(request, ex);
            return ApprovalResult.Failure($"Unexpected error: {ex.Message}",
"UNEXPECTED_ERROR");
        }
    }

    private LoanApprovalRequest MapToGatewayRequest(ApprovalRequest request)
    {
        return new LoanApprovalRequest
        {
            applicantId = request.NationalId,
            nationalId = request.NationalId,
            policyId = request.PolicyId,
            chassisNumber = request.ChassisNumber,
            regionId = request.Region.GetExternalApiCode(),
            plateNumber = request.PlateNumber.GetFullNumber(),
            branchCode = request.Branch.Code,
            loanAmount = request.Amount.Value,
            description = request.Description ?? "Standard approval request",
            processingMode = request.ProcessingMode,
            fromDate = request.ValidFrom,
            toDate = request.ValidTo
        };
    }
}
```

Step 2: Integrate the Beachhead:

```
// Initially, route only Approval through the beachhead
public class LoanProcessingEngine
{
    private readonly ILoanApproval _approval;
    private readonly ILoanCancellation _cancellation; // Still legacy

    public LoanProcessingEngine(ILoanApproval approval, ILoanCancellation
cancellation)
    {
        _approval = approval;
        _cancellation = cancellation;
    }

public async Task ProcessLoan(LoanRequest request)
    {
        if (request.Type == LoanType.Approval)
        {
            // Use the clean beachhead
            var approvalRequest = MapToApprovalRequest(request);
            return await _approval.ProcessAsync(approvalRequest);
        }
        else
```

```
        {
            // Still using legacy
            return await _cancellation.CancelAsync(request);
        }
    }
}
```

Step 3: Expand the Beachhead:

```
// Now you can expand to Cancellation
public interface ILoanCancellation
{
    Task<CancellationResult> ProcessAsync(CancellationRequest request);
}

public class LoanCancellation : ILoanCancellation
{
    // Clean implementation using same patterns
}

// And then to other areas
public interface ILoanReporting { }
public interface ILoanExport { }
public interface ILoanAudit { }
```

Why It Works:

You now have a clean island inside the legacy system. You can:

- **Test the approval workflow independently**
- **Add features to approval without affecting other parts**
- **Use approval as a pattern for future refactoring**
- **Prove the value of refactoring to stakeholders**

The Critical Condition:

The beachhead must be strategically connected to future change. Cleaning a dead utility class isn't a beachhead. Cleaning a heavily changed approval workflow that gives you a stable boundary is.

Strategy 4: Hotspot-First Refactoring

Definition

Hotspot-First refactoring prioritizes refactoring based on pain, not just code quality. Instead of asking "Where is the worst code?" we ask "Where does bad code cause the most pain?"

Core Question

"Where does technical debt actually hurt us?"

When to Use

- **You want to prioritize based on pain, not just code quality**
- **You need to demonstrate value quickly**
- **You have limited time and resources**
- **You need to build momentum for a larger refactoring effort**

When NOT to Use

- **You have clear business priorities that override technical pain**
- **The pain is caused by external factors (not code quality)**
- **The team disagrees on where the pain is**
- **You don't have data to support the decision**

Example: Loan Management System

Hotspot Matrix:

Area	Code Quality	Change Frequency	Business Impact	Pain	Priority
Approval	Bad	High	High	9/10	1
External API	Bad	High	High	8/10	2
Request Building	Bad	Medium	Medium	7/10	3

Cancellation	**Medium**	**High**	**High**	**7/10**	**4**
Excel Reading	**Bad**	**Low**	**Medium**	**5/10**	**5**
Audit Logging	**Good**	**Low**	**Medium**	**3/10**	**6**
Retry Logic	**Medium**	**High**	**High**	**7/10**	**7**

Why It Works:

Even if Excel has uglier code, Approval is probably the better refactoring target because it causes more pain. This is a more mature way of prioritizing technical debt.

PART 3: HOW DO I CROSS BOUNDARIES?

Strategy 5: Sandwich Refactoring

Definition

Sandwich refactoring combines Top-Down and Bottom-Up approaches. You start from the top (business workflow) and the bottom (infrastructure) simultaneously, working toward a clean boundary in the middle.

Core Question

"Can I improve both sides until they meet at a useful boundary?"

When to Use

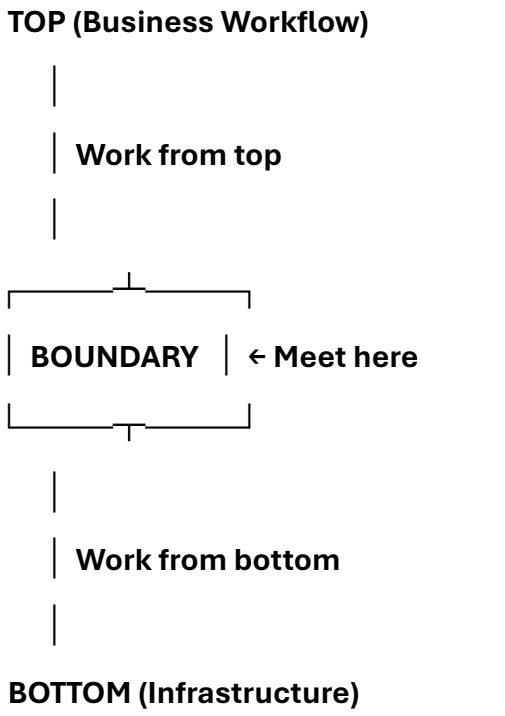
- **Both the high-level workflow and the low-level infrastructure need work**
- **You want to create a seam before making deeper changes**
- **You have the resources to work on both ends**
- **You need to see progress at multiple levels**

When NOT to Use

- **Only one side has problems**
- **You don't understand both layers well**
- **You don't have the resources to work both ends**
- **The boundary is already well-defined**

Example: Loan Management System

The Sandwich Strategy:



Step 1: TOP - Create a seam in the business workflow:

```
// BEFORE: Direct dependency
public class LoanProcessingEngine
{
    private readonly ExternalLoanService _loanService = new
ExternalLoanService();

    public async Task ProcessLoan(LoanRequest request)
    {
        var response = await _loanService.AddLoan(request);
```

```
        // Process response
    }
}

// AFTER: Introduce interface
public interface ILoanGateway
{
    Task<LoanResult> ApproveLoanAsync(LoanApprovalRequest request);
    Task<LoanResult> CancelLoanAsync(LoanCancellationRequest request);
}

public class LoanProcessingEngine
{
    private readonly ILoanGateway _loanGateway;

    public LoanProcessingEngine(ILoanGateway loanGateway)
    {
        _loanGateway = loanGateway;
    }

    public async Task ProcessLoan(LoanRequest request)
    {
        var response = await _loanGateway.ApproveLoanAsync(request);
        // Process response
    }
}
```

Step 2: BOTTOM - Clean the infrastructure:

```
// Extract the retry logic, phone normalization, etc. as shown in Bottom-Up
example
public class ExternalLoanGateway : ILoanGateway
{
    private readonly IHttpClientFactory _httpClientFactory;
    private readonly IRetryHandler _retryHandler;
    private readonly IJsonSerializer _serializer;

    public ExternalLoanGateway(
        IHttpClientFactory httpClientFactory,
        IRetryHandler retryHandler,
        IJsonSerializer serializer)
    {
        _httpClientFactory = httpClientFactory;
        _retryHandler = retryHandler;
        _serializer = serializer;
    }

    public async Task<LoanResult> ApproveLoanAsync(LoanApprovalRequest
request)
    {
        // Clean implementation using extracted components
    }

    public async Task<LoanResult> CancelLoanAsync(LoanCancellationRequest
request)
    {
```

```
        // Clean implementation using extracted components
    }
}
```

Step 3: MIDDLE - Connect them:

```
// Now both sides meet at the boundary
public class LoanProcessingEngine
{
    private readonly ILoanGateway _loanGateway;

    public LoanProcessingEngine(ILoanGateway loanGateway)
    {
        _loanGateway = loanGateway;
    }

    public async Task ProcessLoanAsync(LoanRequest request)
    {
        // Clean workflow using the gateway
        var approvalRequest = BuildApprovalRequest(request);
        var result = await _loanGateway.ApproveLoanAsync(approvalRequest);
        await HandleResultAsync(result);
    }
}
```

Why It Works:

You don't have to clean the entire stack before creating a useful boundary. The application layer can be simplified while the infrastructure implementation can independently be cleaned.

Strategy 6: Seam-First Refactoring

Definition

Seam-First refactoring begins by creating a seam (a place where behavior can be altered without editing the place where the behavior normally occurs) before cleaning any code.

Core Question

"What boundary would make this code changeable?"

When to Use

- **The code is highly coupled**
- **Everything is connected to everything else**
- **You can't safely change anything**
- **You need to enable testing, parallel development, or multiple implementations**

When NOT to Use

- **The code is already loosely coupled**
- **There are already clear boundaries**
- **You don't need the flexibility that a seam provides**

Example: Loan Management System

Problem: LoanProcessingEngine is tightly coupled to ExternalLoanService.

BEFORE:

```
public class LoanProcessingEngine
{
    private readonly ExternalLoanService _loanService = new
ExternalLoanService();

    public async Task ProcessLoan(LoanRequest request)
    {
        var response = await _loanService.ApproveLoanAsync(request);
        // Process response
    }
}
```

SEAM-FIRST - Create the Seam:

```
// Step 1: Create the interface
public interface ILoanGateway
{
    Task<LoanResult> ApproveAsync(LoanApprovalRequest request);
    Task<LoanResult> CancelAsync(LoanCancellationRequest request);
}
```

// Step 2: Create the result model

```
public class LoanResult
{
    public bool IsSuccess { get; }
```

```
    public string Message { get; }
    public string ApplicantId { get; }
    public int? StatusCode { get; }

    public static LoanResult Success(string applicantId, string message =
"Success", int statusCode = 200)
        => new LoanResult(true, message, applicantId, statusCode);

    public static LoanResult Failure(string message, int statusCode = 500)
        => new LoanResult(false, message, null, statusCode);

    private LoanResult(bool isSuccess, string message, string applicantId,
int? statusCode)
    {
        IsSuccess = isSuccess;
        Message = message;
        ApplicantId = applicantId;
        StatusCode = statusCode;
    }
}
```

// Step 3: Implement the gateway

```
public class ExternalLoanGateway : ILoanGateway
{
    private readonly ExternalLoanService _loanService;
    private readonly ILogger _logger;

    public ExternalLoanGateway(ExternalLoanService loanService, ILogger
logger)
    {
        _loanService = loanService;
        _logger = logger;
    }

    public async Task<LoanResult> ApproveAsync(LoanApprovalRequest request)
    {
        try
        {
            _logger.LogDebug($"Approving loan for: {request.applicantId}");
            var response = await _loanService.ApproveLoanAsync(request);

            if (response.IsSuccessful)
            {
                return LoanResult.Success(
                    response.Data.applicantId,
                    response.Data.responseMessage,
                    response.StatusCode
                );
            }

            return LoanResult.Failure(response.ErrorMessage,
response.StatusCode);
        }
```

```
        catch (Exception ex)
        {
            _logger.LogError($"Approval failed: {ex.Message}");
            return LoanResult.Failure($"Approval failed: {ex.Message}");
        }
    }

    // Similar for Cancel
}
```

// Step 4: Inject into LoanProcessingEngine

```
public class LoanProcessingEngine
{
    private readonly ILoanGateway _loanGateway;

    public LoanProcessingEngine(ILoanGateway loanGateway)
    {
        _loanGateway = loanGateway;
    }
}
```

What This Enables:

1. **Testing:**

```
var mock = new Mock<ILoanGateway>();
mock.Setup(x => x.ApproveAsync(It.IsAny<LoanApprovalRequest>()))
    .ReturnsAsync(LoanResult.Success("123"));

var engine = new LoanProcessingEngine(mock.Object);
```

// Now we can test the engine without calling the real API!

2. **Parallel Development:**

- **Team A can work on improving the engine**
- **Team B can work on improving the gateway**
- **Independent work!**

3. **Multiple Implementations:**

```
public class ExternalLoanGateway : ILoanGateway { }  // Production
public class MockLoanGateway : ILoanGateway { }      // Testing
public class CachedLoanGateway : ILoanGateway { }    // Performance
public class LoggingLoanGateway : ILoanGateway { }   // Logging
```

Why It Works:

You haven't necessarily improved all the code. But you've improved your ability to change the code. Sometimes your first refactoring isn't about making the code cleaner. It's about making the code changeable.

Strategy 7: Strangler Refactoring

Definition

Strangler refactoring gradually replaces a legacy capability without rewriting everything at once. New functionality is built alongside the old, and gradually, the old functionality is routed to the new implementation until the old can be removed.

Core Question

"Can I gradually replace a legacy capability without rewriting everything?"

When to Use

- **The system is large and complex**
- **You can't change everything at once**
- **You need to maintain the existing system while building the new one**
- **You need to prove the new approach works before committing fully**

When NOT to Use

- **The system is small enough to refactor directly**
- **You have the resources for a full rewrite**
- **The current system is completely broken**

Example: Loan Management System

Phase 1: Route Approval to New Implementation

```
public class LoanRouter
{
    private readonly ILoanProcessor _newLoanProcessor;
    private readonly ILoanProcessor _legacyLoanProcessor;
```

```
    public LoanRouter(ILoanProcessor newLoanProcessor, ILoanProcessor
legacyLoanProcessor)
    {
        _newLoanProcessor = newLoanProcessor;
        _legacyLoanProcessor = legacyLoanProcessor;
    }

    public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
    {
        if (request.Type == LoanType.Approval)
        {
            // New implementation
            return await _newLoanProcessor.ProcessAsync(request);
        }
        else
        {
            // Legacy implementation
            return await _legacyLoanProcessor.ProcessAsync(request);
        }
    }
}
```

Phase 2: Route Cancellation to New Implementation

```
public class LoanRouter
{
    public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
    {
        if (request.Type == LoanType.Approval)
        {
            return await _newLoanProcessor.ProcessAsync(request);
        }
        else if (request.Type == LoanType.Cancellation)
        {
            return await _newLoanProcessor.ProcessAsync(request);
        }
        else
        {
            return await _legacyLoanProcessor.ProcessAsync(request);
        }
    }
}
```

Phase 3: Route Everything to New Implementation

```
public class LoanRouter
{
    public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
    {
        return await _newLoanProcessor.ProcessAsync(request);
    }
}
```

Phase 4: Remove Legacy Code

// Delete the legacy loan processor

// Delete the LoanRouter

// The new implementation is now the only implementation

Why It Works:

You can gradually replace the system while maintaining the existing behavior. Each phase is safe and incremental. You can roll back at any point if issues are discovered.

Strategy 8: Branch-by-Abstraction (Parallel Refactoring)

Definition

Branch-by-Abstraction is a technique for making large-scale changes by introducing an abstraction layer that both the old and new implementations can work behind, then migrating clients from the old to the new.

Core Question

"Can I introduce a stable abstraction and migrate implementations behind it?"

When to Use

- **The implementation is large**
- **Many callers exist**
- **The replacement will take time**
- **You need incremental migration**

When NOT to Use

- **The change is small enough to do directly**
- **There are no callers to migrate**
- **You don't have the resources for parallel implementations**

Example: Loan Management System

Step 1: Introduce the Abstraction

```
// BEFORE: Direct dependency
public class LoanProcessingEngine
{
    private readonly ExternalLoanService _loanService = new
ExternalLoanService();
}

// AFTER: Abstraction
public interface ILoanProcessor
{
    Task<LoanResult> ProcessAsync(LoanRequest request);
}

public class LoanProcessingEngine
{
    private readonly ILoanProcessor _loanProcessor;
}
```

Step 2: Create Both Implementations

```
public class LegacyLoanProcessor : ILoanProcessor
{
    private readonly ExternalLoanService _legacyService;

    public async Task<LoanResult> ProcessAsync(LoanRequest request)
    {
        // Legacy implementation
        var response = await _legacyService.ExecuteLoanOperation(request, 1);
        return MapToResult(response);
    }
}

public class NewLoanProcessor : ILoanProcessor
{
    private readonly INewLoanGateway _newGateway;
    private readonly ILogger _logger;

    public async Task<LoanResult> ProcessAsync(LoanRequest request)
    {
        // New implementation
        _logger.LogInfo($"Processing with new implementation");
        var result = await
_newGateway.ApproveAsync(MapToGatewayRequest(request));
        return MapToResult(result);
    }
}
```

Step 3: Use Feature Flags

```
public class FeatureFlaggedLoanProcessor : ILoanProcessor
{
    private readonly LegacyLoanProcessor _legacy;
    private readonly NewLoanProcessor _new;
    private readonly IFeatureFlagService _featureFlagService;
```

```
    public async Task<LoanResult> ProcessAsync(LoanRequest request)
    {
        if (_featureFlagService.IsEnabled("UseNewLoanProcessor"))
        {
            return await _new.ProcessAsync(request);
        }
        return await _legacy.ProcessAsync(request);
    }
}
```

Step 4: Gradually Enable

```
// Start with 1% of users
// Monitor for errors
// Increase to 10%
// Monitor for errors
// Increase to 50%
// Monitor for errors
// Increase to 100%
// Remove the legacy implementation
```

Why It Works:

You can safely replace a large implementation without a big-bang rewrite. Each step is reversible and monitored. You can roll back at any point.

PART 4: HOW DO I MOVE THROUGH A METHOD?

Strategy 9: Deepest-First Refactoring

Definition

Deepest-First refactoring starts at the deepest nested blocks and works outward. The goal is to reduce local complexity by extracting cohesive inner blocks.

Core Question

"Can I eliminate cohesive nested regions from the inside out?"

When to Use

- **The method has deep nesting**
- **Inner blocks are cohesive**

- **Inner blocks have clear inputs and outputs**
- **You want to reduce indentation immediately**

When NOT to Use

- **Inner blocks are not cohesive**
- **The nesting is not the main problem**
- **The inner blocks don't have clear boundaries**

Example: Loan Management System

BEFORE:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    if (request.IsValid)
    {
        if (request.Type == LoanType.Approval)
        {
            if (request.Amount > 100000)
            {
                if (request.Customer.IsVip)
                {
                    // Process VIP high-value approval (40 lines)
                    var creditCheck = await
_creditService.CheckAsync(request.Customer);
                    if (creditCheck.Passed)
                    {
                        var approval = await
_approvalService.ApproveAsync(request);
                        if (approval.Success)
                        {
                            var notification = await
_notificationService.SendAsync(request.Customer, "Approved");
                            await _auditService.LogAsync(request, approval,
notification);
                            return LoanResult.Success(approval);
                        }
                        else
                        {
                            await _auditService.LogAsync(request, approval,
"ApprovalFailed");
                            return LoanResult.Failure("ApprovalFailed");
                        }
                    }
                    else
                    {
                        await _auditService.LogAsync(request, creditCheck,
"CreditCheckFailed");
                        return LoanResult.Failure("CreditCheckFailed");
                    }
```

```
                }
                else
                {
                    // Process regular high-value approval (30 lines)
                }
            }
            else
            {
                // Process standard approval (50 lines)
            }
        }
        else
        {
            // Process cancellation (40 lines)
        }
    }
    else
    {
        return LoanResult.Failure("Invalid request");
    }
}
```

DEEPEST-FIRST - Start with the deepest blocks:

Step 1: Extract the VIP high-value approval (the deepest branch):

```
private async Task<LoanResult> ProcessVipHighValueApprovalAsync(LoanRequest
request)
{
    var creditCheck = await _creditService.CheckAsync(request.Customer);
    if (!creditCheck.Passed)
    {
        await _auditService.LogAsync(request, creditCheck,
"CreditCheckFailed");
        return LoanResult.Failure("CreditCheckFailed");
    }

    var approval = await _approvalService.ApproveAsync(request);
    if (!approval.Success)
    {
        await _auditService.LogAsync(request, approval, "ApprovalFailed");
        return LoanResult.Failure("ApprovalFailed");
    }

    var notification = await _notificationService.SendAsync(request.Customer,
"Approved");
    await _auditService.LogAsync(request, approval, notification);
    return LoanResult.Success(approval);
}
```

Step 2: Extract the regular high-value approval:

```
private async Task<LoanResult>
ProcessRegularHighValueApprovalAsync(LoanRequest request)
```

```
{
    // 30 lines extracted
}
```

Step 3: Extract the standard approval:

```
private async Task<LoanResult> ProcessStandardApprovalAsync(LoanRequest
request)
{
    // 50 lines extracted
}
```

Step 4: Extract the cancellation:

private async Task<LoanResult> ProcessCancellationAsync(LoanRequest request)

{

// 40 lines extracted

}

AFTER:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    if (!request.IsValid)
        return LoanResult.Failure("Invalid request");

    if (request.Type == LoanType.Approval)
    {
        if (request.Amount > 100000)
        {
            return request.Customer.IsVip
                ? await ProcessVipHighValueApprovalAsync(request)
                : await ProcessRegularHighValueApprovalAsync(request);
        }
        return await ProcessStandardApprovalAsync(request);
    }

    return await ProcessCancellationAsync(request);
}
```

Why It Works:

Deep branches often have high local cohesion and clear inputs and outputs. They're frequently easy extraction candidates. More importantly, extracting them reduces indentation immediately.

Strategy 10: Outer-First Refactoring

Definition

Outer-First refactoring starts at the outer structure and works inward. The goal is to simplify the control-flow structure by removing nesting.

Core Question

"Can I simplify the control-flow structure from the outside in?"

When to Use

- **Nesting obscures the happy path**
- **You want to reveal the main workflow first**
- **You want to use guard clauses for exceptional cases**

When NOT to Use

- **The nesting is not the main problem**
- **The outer structure is not clear**
- **You need to preserve complex branching logic**

Example: Loan Management System

BEFORE:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    if (request.IsValid)
    {
        if (request.Type == LoanType.Approval)
        {
            // Process approval
        }
        else
        {
            // Process cancellation
        }
    }
    else
    {
        return LoanResult.Failure("Invalid request");
    }
```

```
}
```

OUTER-FIRST - Start with the outer structure:

Step 1: Use guard clauses for the invalid case:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    // Guard clause
    if (!request.IsValid)
        return LoanResult.Failure("Invalid request");

    if (request.Type == LoanType.Approval)
    {
        // Process approval
    }
    else
    {
        // Process cancellation
    }
}
```

Step 2: Extract the branches:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    if (!request.IsValid)
        return LoanResult.Failure("Invalid request");

    return request.Type == LoanType.Approval
        ? await ProcessApprovalAsync(request)
        : await ProcessCancellationAsync(request);
}
```

Step 3: Further simplify:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    Guard.Against.Invalid(request);

    return request.Type == LoanType.Approval
        ? await ProcessApprovalAsync(request)
        : await ProcessCancellationAsync(request);
}
```

AFTER:

```
public async Task<LoanResult> ProcessLoanAsync(LoanRequest request)
{
    // Guard clauses - remove nesting
    if (!request.IsValid)
        return LoanResult.Failure("Invalid request");

    if (request.Type == LoanType.Approval)
        return await ProcessApprovalAsync(request);
```

```
    return await ProcessCancellationAsync(request);
}
```

Why It Works:

You've reduced the nesting dramatically. The happy path (valid request, approval) is now visible. Remove exceptional or negative paths from the main path so the happy path becomes visible.

Strategy 11: Data-Flow (Data-Transformation) Refactoring

Definition

Data-Flow refactoring follows the data transformation through the method, extracting around semantic transformations rather than indentation.

Core Question

"Can I separate the transformations happening inside this method?"

When to Use

- **The method performs multiple data transformations**
- **You want to reveal the transformation pipeline**
- **Following data flow is clearer than following control flow**

When NOT to Use

- **The method doesn't have clear data transformations**
- **Control flow is the main problem**
- **The transformations are tightly coupled**

Example: Loan Management System

BEFORE:

```
public async Task ProcessLoanRequestsAsync()
```

```
{
    // Transformation 1: Excel row to LoanRequestRecord
    var allRecords = ReadExcelData(_loanApprovalExcelPath,
RequestType.Approval);

    // Transformation 2: Filter records
    var processedApplicants = await GetProcessedApplicantsAsync();
    var retryTracker = await GetRetryTrackerAsync();
    var recordsToProcess = allRecords
        .Where(r => !processedApplicants.Contains(r.ChassisNumber))
        .ToList();

    // Transformation 3: Transform to API request
    var requests = recordsToProcess
        .Select(r => BuildRequestFromExcel(r, RequestType.Approval))
        .ToList();

    // Transformation 4: Process API requests
    var results = new List<ProcessingResult>();
    foreach (var request in requests)
    {
        var response = await _loanService.ExecuteLoanOperation(request, 1);
        results.Add(ProcessResponse(response));
    }

    // Transformation 5: Update tracking
    await UpdateRetryTrackerAsync(results.Where(r => !r.IsSuccess));
    await MarkApplicantsAsProcessedAsync(results.Where(r =>
r.IsSuccess).Select(r => r.ChassisNumber));


    // Transformation 6: Archive failures
    await ArchiveToDeadLetterAsync(results.Where(r => !r.IsSuccess),
RequestType.Approval);
}
```

DATA-FLOW - Extract around semantic transformations:

```
public async Task ProcessLoanRequestsAsync()
{
    // 1. Load → Excel to Records
    var records = await LoadRecordsFromExcelAsync(RequestType.Approval);

    // 2. Filter → Records to Processable Records
    var filteredRecords = await FilterRecordsAsync(records);

    // 3. Build → Records to API Requests
    var requests = BuildApiRequests(filteredRecords);

    // 4. Process → API Requests to Results
    var results = await ProcessApiRequestsAsync(requests);

    // 5. Update → Results to Tracking
    await UpdateTrackingAsync(results);
```

```
    // 6. Archive → Failed Results to Dead Letter
    await ArchiveFailuresAsync(results);

    // 7. Report
    DisplaySummary(results);
}

// Each transformation is now a separate, focused method
private async Task<List<LoanRequestRecord>>
LoadRecordsFromExcelAsync(RequestType type)
{
    return ReadExcelData(GetExcelPath(type), type);
}

private async Task<List<LoanRequestRecord>>
FilterRecordsAsync(List<LoanRequestRecord> records)
{
    var processed = await GetProcessedApplicantsAsync();
    var retryTracker = await GetRetryTrackerAsync();

    return records
        .Where(r => !processed.Contains(r.ChassisNumber))
        .Where(r => !retryTracker.TryGetValue(r.ChassisNumber, out var count)
|| count < _maxRetryAttempts)
        .ToList();
}

private List<ExternalLoanRequest> BuildApiRequests(List<LoanRequestRecord>
records)
{
    return records
        .Select(r => BuildRequestFromExcel(r, RequestType.Approval))
        .ToList();
}

private async Task<List<ProcessingResult>>
ProcessApiRequestsAsync(List<ExternalLoanRequest> requests)
{
    var results = new List<ProcessingResult>();
    foreach (var request in requests)
    {
        var response = await _loanService.ExecuteLoanOperation(request, 1);
        results.Add(ProcessResponse(response));
    }
    return results;
}
```

Why It Works:

The data flow becomes visible. Each transformation is a clear step in the pipeline. The method reads like: Load → Filter → Build → Process → Update → Archive → Report.

Strategy 12: Responsibility-First Refactoring

Definition

Responsibility-First refactoring moves knowledge to the place where it belongs by identifying which lines of code require knowledge that doesn't belong to the current class.

Core Question

"Which line of this method requires knowledge of something that doesn't belong here?"

When to Use

- **The method knows too much about other components**
- **Feature envy is present**
- **Responsibilities are mixed**

When NOT to Use

- **The knowledge does belong in the current class**
- **Moving the knowledge would create more coupling**
- **The responsibility boundary is unclear**

Example: Loan Management System

Problem: BuildRequestFromExcel knows about Excel columns, region mapping, plate numbers, and external request structure.

BEFORE:

```
public ExternalLoanRequest BuildRequestFromExcel(LoanRequestRecord record,
RequestType requestType)
{
    try
    {
        var request = new ExternalLoanRequest();
        var isSpecialRegion = record.RegionId == 1 || record.RegionId == 31;

        if (requestType == RequestType.Approval)
        {
            var approvalRequest = new LoanApprovalRequest
            {
```

```
                branchIds = null,
                description = record.Description ?? "Standard loan approval
request",
                branchCode = record.BranchId.ToString("D2"),
                nationalId = record.NationalId,
                reason = record.ReasonCode ?? "01",
                requestDate = DateTime.Now,
                applicantId = record.NationalId?.Trim(),
                organizationId = record.OrganizationId,
                organizationDescription = record.OrganizationDescription ??
"General applicant",
                phoneNumber = null,
                contractNumber = null,
                fromDate = record.ValidFromDate,
                toDate = record.ValidToDate,
                manufactureYear = null,
                policyId = record.PolicyId,
                chassisNumber = record.ChassisNumber?.Trim(),
                vehicleType = 0,
                processingMode = record.ProcessingMode,
                loanAmount = record.LoanAmount,
                regionId = isSpecialRegion ? "32" :
record.RegionId.ToString("D2")
            };

            if (record.ImportStatus == 1)
            {
                approvalRequest.importPlateDigits =
record.ImportDigits?.ToString();
            }
            else
            {
                approvalRequest.plateNumber =
$"{record.PlateRegionCode}{record.PlateFirstPart}{record.PlateLetter}{record.
PlateSecondPart}";
            }

            request.ApprovalRequest = approvalRequest;
        }
        else
        {
            // Similar logic for cancellation
        }

        return request;
    }
    catch (Exception ex)
    {
        LogError($"Error building request from Excel: {ex.Message}");
        return null;
    }
}
```

RESPONSIBILITY-FIRST - Move knowledge to the right place:

Step 1: Move region logic to a Region class:

```
public class Region
{
    public int Id { get; }
    public string Name { get; }

    public Region(int id, string name)
    {
        Id = id;
        Name = name;
    }

    public string GetExternalApiCode()
    {
        // Special regions map to 32 in the external API
        if (Id == 1 || Id == 31)
            return "32";

        return Id.ToString("D2");
    }

    public bool IsSpecialRegion() => Id == 1 || Id == 31;
}

// Usage in LoanRequestRecord
public class LoanRequestRecord
{
    public Region Region { get; set; }
    // ...
}
```

Step 2: Move plate number logic to a PlateNumber class:

```
public class PlateNumber
{
    public string RegionCode { get; }
    public string FirstPart { get; }
    public string Letter { get; }
    public string SecondPart { get; }
    public bool IsImported { get; }
    public string ImportDigits { get; }

    public PlateNumber(string regionCode, string firstPart, string letter,
string secondPart)
    {
        RegionCode = regionCode;
        FirstPart = firstPart;
        Letter = letter;
        SecondPart = secondPart;
        IsImported = false;
    }

    public PlateNumber(string importDigits)
```

```
    {
        ImportDigits = importDigits;
        IsImported = true;
    }

    public string GetFullNumber()
    {
        if (IsImported)
            return ImportDigits;

        return $"{RegionCode}{FirstPart}{Letter}{SecondPart}";
    }
}

// Usage in LoanRequestRecord
public class LoanRequestRecord
{
    public PlateNumber PlateNumber { get; set; }
    // ...
}
```

Step 3: Move request building to dedicated builders:

```
public class LoanApprovalRequestBuilder
{
    public LoanApprovalRequest Build(LoanRequestRecord record)
    {
        return new LoanApprovalRequest
        {
            branchIds = null,
            description = record.Description ?? "Standard loan approval
request",
            branchCode = record.BranchId.ToString("D2"),
            nationalId = record.NationalId,
            reason = record.ReasonCode ?? "01",
            requestDate = DateTime.Now,
            applicantId = record.NationalId?.Trim(),
            organizationId = record.OrganizationId,
            organizationDescription = record.OrganizationDescription ??
"General applicant",
            phoneNumber = null,
            contractNumber = null,
            fromDate = record.ValidFromDate,
            toDate = record.ValidToDate,
            manufactureYear = null,
            policyId = record.PolicyId,
            chassisNumber = record.ChassisNumber?.Trim(),
            vehicleType = 0,
            processingMode = record.ProcessingMode,
            loanAmount = record.LoanAmount,
            regionId = record.Region.GetExternalApiCode(),
            plateNumber = record.PlateNumber.GetFullNumber(),
            importPlateDigits = record.PlateNumber.IsImported ?
record.PlateNumber.ImportDigits : null
        };
```

```
    }
}

public class LoanCancellationRequestBuilder
{
    public LoanCancellationRequest Build(LoanRequestRecord record)
    {
        return new LoanCancellationRequest
        {
            plateNumber = record.PlateNumber.IsImported ? null :
record.PlateNumber.GetFullNumber(),
            importPlateDigits = record.PlateNumber.IsImported ?
record.PlateNumber.ImportDigits : null,
            description = record.Description ?? "Loan cancellation request",
            reason = record.ReasonCode ?? "01",
            requestDate = DateTime.Now,
            applicantId = record.NationalId?.Trim(),
            organizationId = record.OrganizationId,
            organizationDescription = record.OrganizationDescription ??
"General applicant",
            phoneNumber = null,
            contractNumber = null,
            fromDate = DateTime.Now,
            toDate = DateTime.Now.AddYears(1),
            manufactureYear = null,
            policyId = record.PolicyId,
            chassisNumber = record.ChassisNumber?.Trim(),
            vehicleType = 0,
            loanAmount = record.LoanAmount
        };
    }
}
```

Step 4: Clean BuildRequestFromExcel:

```
public ExternalLoanRequest BuildRequestFromExcel(LoanRequestRecord record,
RequestType requestType)
{
    try
    {
        var request = new ExternalLoanRequest();

        if (requestType == RequestType.Approval)
        {
            var builder = new LoanApprovalRequestBuilder();
            request.ApprovalRequest = builder.Build(record);
        }
        else
        {
            var builder = new LoanCancellationRequestBuilder();
            request.CancellationRequest = builder.Build(record);
        }

        return request;
```

```
    }
    catch (Exception ex)
    {
        LogError($"Error building request from Excel: {ex.Message}");
        return null;
    }
}
```

Why It Works:

Each piece of knowledge now lives in the right place. Region logic is in Region. Plate number logic is in PlateNumber. Request building is in dedicated builders. The method no longer knows about Excel column names, region mapping, or plate number formatting.

Strategy 13: Stepwise Extraction

Definition

Stepwise extraction is the simplest strategy: extract the method step by step, one logical block at a time.

Core Question

"Can I extract the method step by step?"

When to Use

- **The method is long but has clear logical steps**
- **You want to make incremental progress**
- **You want to keep changes small and safe**

When NOT to Use

- **The method doesn't have clear logical steps**
- **You need a more strategic approach**
- **The method is already well-structured**

Example: Loan Management System

Step by step extraction:

```
// STEP 1: Identify the first logical block
public async Task ProcessLoanRequestsAsync()
{
    // BLOCK 1: Setup and logging
    LogStep("Starting Loan Approval Request Processing");
    LogInfo($"Excel file path: {_loanApprovalExcelPath}");

    // BLOCK 2: Load and filter records
    // ... rest of method
}

// STEP 2: Extract block 1
private void SetupLogging()
{
    LogStep("Starting Loan Approval Request Processing");
    LogInfo($"Excel file path: {_loanApprovalExcelPath}");
}

// STEP 3: Identify block 2
public async Task ProcessLoanRequestsAsync()
{
    SetupLogging();

    // BLOCK 2: Load and filter records
    // ... rest of method
}

// Continue extracting one block at a time
```

Strategy 14: Guard-Clause Simplification

Definition

Guard-clause simplification replaces complex conditionals with early returns for exceptional cases, making the happy path visible.

Core Question

"Can I use early returns to remove nesting?"

When to Use

- **The method has deep nesting with exceptional cases**
- **You want to reveal the happy path**
- **You want to reduce cognitive complexity**

When NOT to Use

- **The exceptional cases are not clear**
- **You need to preserve complex branching logic**
- **The method has multiple exit points that would be confusing**

Example: Loan Management System

BEFORE:

```
public async Task ProcessRecord(LoanRequestRecord record)
{
    if (record != null)
    {
        if (!string.IsNullOrEmpty(record.ChassisNumber))
        {
            if (!string.IsNullOrEmpty(record.NationalId))
            {
                if (record.LoanAmount > 0)
                {
                    // Process the record - HAPPY PATH
                    await ProcessValidRecord(record);
                }
                else
                {
                    LogError("Invalid loan amount");
                }
            }
            else
            {
                LogError("Invalid National ID");
            }
        }
        else
        {
            LogError("Invalid Chassis Number");
        }
    }
    else
    {
        LogError("Record is null");
    }
}
```

AFTER - Guard Clauses:

```
public async Task ProcessRecord(LoanRequestRecord record)
{
    // Guard clauses - handle exceptional cases first
    if (record == null)
    {
        LogError("Record is null");
        return;
```

```
    }

    if (string.IsNullOrEmpty(record.ChassisNumber))
    {
        LogError("Invalid Chassis Number");
        return;
    }

    if (string.IsNullOrEmpty(record.NationalId))
    {
        LogError("Invalid National ID");
        return;
    }

    if (record.LoanAmount <= 0)
    {
        LogError("Invalid loan amount");
        return;
    }

    // Happy path is now visible and not nested
    await ProcessValidRecord(record);
}
```

PART 5: HOW DO I CONTROL RISK?

Strategy 15: Characterization-First Refactoring

Definition

Characterization-First refactoring begins by writing tests that capture what the system currently does (not what it should do) before making any changes.

Core Question

"How do I protect behavior before changing it?"

When to Use

- **There are no tests**
- **The code is poorly understood**
- **The behavior is complex or subtle**

- **You need to preserve current behavior**

When NOT to Use

- **There are already good tests**
- **The behavior is well understood**
- **The behavior is clearly wrong and needs to change**

Example: Loan Management System

Problem: We don't know what these numbers mean:

```
if (regionId == 1 || regionId == 31)
    regionId = 32;
```

CHARACTERIZATION TEST:

```
[Fact]
public void RegionMapper_ShouldMapSpecialRegions()
{
    // Arrange
    var engine = new LoanProcessingEngine();

    // Act & Assert - Test current behavior
    // We don't know if this is correct, but it's what the system does
    var result1 = engine.MapRegion(1);
    Assert.Equal(32, result1);  // Preserve this behavior

    var result2 = engine.MapRegion(31);
    Assert.Equal(32, result2);  // Preserve this behavior

    var result3 = engine.MapRegion(32);
    Assert.Equal(32, result3);  // Preserve this behavior

    var result4 = engine.MapRegion(5);
    Assert.Equal(5, result4);   // Preserve this behavior

    var result5 = engine.MapRegion(99);
    Assert.Equal(99, result5);  // Preserve this behavior
}
```

Why It Works:

The test does not say "31 is the correct business value." It says "31 is what the system currently does." Once the behavior is captured, we can safely refactor the region mapping.

Strategy 16: Change-Slicing

Definition

Change-slicing breaks large changes into tiny, safe pieces that can be verified independently.

Core Question

"Can I break the change into tiny, safe pieces?"

When to Use

- **The change is large and risky**
- **You want to reduce risk**
- **You want to make each change independently verifiable**

When NOT to Use

- **The change is already small**
- **The pieces are not independently useful**
- **You have high confidence in the change**

Example: Loan Management System

Instead of one large change:

Change 1: Rename ProcessLoanRequestsAsync to ProcessLoanRequests

Change 2: Extract PrepareRecords method

Change 3: Extract ProcessRecords method

Change 4: Extract ReportResults method

Change 5: Update error handling

Change 6: Update logging

Each change is small, safe, and independently verifiable.

Strategy 17: Incremental Migration

Definition

Incremental migration gradually moves from old to new, maintaining both implementations during the transition.

Core Question

"Can I move gradually from old to new?"

When to Use

- **The replacement is large**
- **You can't change everything at once**
- **You need to maintain the system during the transition**

When NOT to Use

- **The change is small enough to do directly**
- **You have the resources for a full migration**
- **The old implementation is broken**

Example: Loan Management System

Phase 1: Use feature flag

```
public class LoanProcessor
{
    private readonly LegacyLoanProcessor _legacy;
    private readonly NewLoanProcessor _new;

    public async Task<LoanResult> ProcessAsync(LoanRequest request)
    {
        if (FeatureFlags.UseNewLoanProcessor)
        {
            return await _new.ProcessAsync(request);
        }
        return await _legacy.ProcessAsync(request);
    }
}
```

Phase 2: Gradually enable for certain conditions

```
public class LoanProcessor
{
    public async Task<LoanResult> ProcessAsync(LoanRequest request)
    {
        // Start with 1% of users
        if (request.Id % 100 < 1)
        {
            return await _new.ProcessAsync(request);
        }
        return await _legacy.ProcessAsync(request);
    }
}
```

Phase 3: Increase percentage

// 10%, then 25%, then 50%, then 75%, then 100%

Phase 4: Remove legacy implementation

Strategy 18: Golden-Master Driven Refactoring

Definition

Golden-Master driven refactoring uses captured outputs from the current system as a baseline for verifying that behavior is preserved during refactoring.

Core Question

"How do I verify the whole system still works?"

When to Use

- **The system processes files and produces outputs**
- **You have known inputs**
- **You want to verify behavior at the system level**

When NOT to Use

- **The system has non-deterministic behavior**
- **The outputs are not stable**
- **You need more granular testing**

Example: Loan Management System

Step 1: Capture Baseline Output

```
[Fact]
public void CaptureGoldenMaster()
{
    // Arrange
    var engine = new LoanProcessingEngine();
    var inputFile = "test_loan_requests.xlsx";

    // Act
    var output = engine.CaptureOutput(inputFile);

    // Store as golden master
    File.WriteAllText("golden_master.json",
        JsonConvert.SerializeObject(output, Formatting.Indented));
}
```

Step 2: Verify Against Golden Master

```
[Fact]
public void VerifyAgainstGoldenMaster()
{
    // Arrange
    var engine = new LoanProcessingEngine();
    var inputFile = "test_loan_requests.xlsx";
    var expected = File.ReadAllText("golden_master.json");
    var expectedOutput = JsonConvert.DeserializeObject<Output>(expected);

    // Act
    var actual = engine.CaptureOutput(inputFile);

    // Assert
    Assert.Equal(expectedOutput.SuccessCount, actual.SuccessCount);
    Assert.Equal(expectedOutput.FailureCount, actual.FailureCount);
    Assert.Equal(expectedOutput.ProcessedApplicants,
actual.ProcessedApplicants);
    Assert.Equal(expectedOutput.AuditLogs, actual.AuditLogs);
    Assert.Equal(expectedOutput.DeadLetters, actual.DeadLetters);
}
```

What Golden Master Covers:

- **Which applicants are marked as processed**
- **Which applicants end up in dead letter**
- **Audit logs content**

- **Retry attempts**
- **Final statuses**